Software Requirements Specification (SRS)

Quiz Application

Version: 1.0

Project Type: Python Mini Project 2

Frontend: Tkinter

Backend: Python

Storage: JSON File

Document Control

Prepared by: Md. Anjir Jaman

Date: 05-09-2026

Version: 1.0

Contents

1. Introduction

1.1 Purpose

The Quiz Application is a Python program designed to test a user's knowledge through multiple-choice questions.

The application shall provide a GUI interface through which users can take a quiz, view their results, and review past score history.

Quiz questions shall be loaded from a local JSON file, and each attempt's score shall be stored in a local JSON file so that history remains available after the application is closed.

1.2 Scope

The system shall provide the following main functions:

- Load quiz questions from a file
- Present questions and multiple-choice options dynamically within a desktop GUI
- Calculate the score based on the user's answers
- Display the results at the end of the quiz
- Append each completed quiz attempt to a local score history file (scores.json)
- Display past quiz attempts inside a scrollable history screen.
- Validate selections using modal warning dialogs before proceeding
- Provide confirmation prompts when attempting to exit an active quiz session

1.3 Intended User

The system is intended for a user who wants to take a short quiz and track their scores over time. The user does not need advanced technical knowledge to operate the application beyond running a Python script.

2. Overall Description

2.1 Product Perspective

The Quiz Application shall operate as a standalone Python GUI program made with Tkinter. The system shall use local JSON files for question data and score storage. No external database, browser, or internet connection is required.

Basic system flow:

User → Python Program → JSON files (questions. Json, score_history.json)

Figure 1: System Architecture

The program shall read questions from a file and display them in the console. The user shall respond via keyboard input. The backend shall process the answer, calculate the score, and read/write the score history file.

2.2 Main Features

- Main Menu
- Start Quiz
- Calculate Score
- View Results
- Save Score History
- View Score History

2.3 Operating Environment

The application shall require:

- A computer
- Python 3
- A terminal or console to run the program
- A Python development environment such as VS Code, PyCharm, or IDLE
- Local file storage

3. Functional Requirements

FR-01: Load Questions

The system shall load quiz questions from a local JSON file at startup.

Each question shall include the question text, a list of options, and the correct answer.

If the question file does not exist or is invalid, the system shall display an error message and exit gracefully.

FR-02: Run Quiz

The system shall present questions one by one. Each option shall be rendered as a clickable Radio button displaying the option's text.

The user's choice shall be stored dynamically

FR-03: Calculate Score

The system shall compare the user’s answers against the correct answers.

The system shall calculate the total number of correct answers and the score as a percentage.

FR-04: Display Results

Upon quiz completion, the system shall display the score, total questions, percentage, and a color-coded status indicator (Green for "Passed", Red for "Needs Improvement").

Action buttons shall allow the user to retake the quiz, view history, or return to the main menu.

FR-05: Save Score History

The system shall save each quiz attempt (name, date, score, total) to a local JSON file.

The score history file shall be appended to, not overwritten, after each attempt.

FR-06: View Score History

The system shall allow the user to view a list of previous quiz attempts from the score history file.

If no score history exists, the system shall display an appropriate message.

FR-07: Input Validation

The system shall validate that the user's answer is one of the valid option letters.

The system shall reject invalid input and re-prompt the user without crashing.

FR-08: Navigation

The system shall provide a main menu allowing the user to move between:

- Start Quiz
- View Score History
- Exit

The user shall be able to return to the main menu after completing a quiz or viewing history.

4. Data Requirements

Each quiz question record shall contain:

Question Record (questions.json):

- question (String, Required): The question prompt text.
- options (Array of Strings, Required): List of possible answer choices.
- answer (String, Required): The exact string value matching the correct choice.

Score History Record (scores.json):

- date (String, Required): Timestamp of completion formatted as YYYY-MM-DD HH:MM:SS.
- score (Integer, Required): Number of correctly answered questions.
- total (Integer, Required): Total number of questions in the quiz.
- percentage (Float, Required): Score percentage rounded to one decimal place.

5. User Interface Requirements

The application shall use a text-based console interface.

5.1 Main Menu

The main menu shall provide access to the main functions of the application: Start Quiz, View Score History, and Exit.

5.2 Quiz Screen

The screen shall display one question at a time.

There shall be 4 options to chose fro,

It shall prompt the user to choose their correct answer.

5.3 Results Screen

The screen shall display the total questions, number of correct answers, and final score.

Example:

Total Questions	Correct Answers	Score
5	4	80%

5.4 Score History Screen

The screen shall display previous quiz attempts in a simple list format, showing date, score, and total questions.

6. Backend Requirements

The Python backend shall:

- Load questions from the question file
- Present questions and capture user input from the console
- Validate submitted answers
- Calculate the score
- Read from and write to the score history file
- Return appropriate results and messages to the console

The backend should keep quiz logic, scoring logic, and file handling in separate functions where practical.

7. Data Storage Requirements

The system shall use JSON for local data storage.

Example questions.json:

```
[
 {
  "question": "Capital of Bangladesh?",
  "options": ["Dhaka", "Chittagong", "Khulna", "Sylhet"],
  "answer": "Dhaka"
 }
]": "Dhaka"}]
```

Example score_history.json:

```
[
 {
```

"date": "2026-09-05 10:15:30",

"score": 4,

"total": 5,

"percentage": 80.0

}]

The system shall support:

- Reading questions
- Appending score history records
- Persistent storage across sessions

8. Non-Functional Requirements

NFR-01: Usability

The console interface should be simple, clear, and easy to follow.

NFR-02: Performance

Normal operations should complete within a reasonable time for the expected small number of questions.

NFR-03: Reliability

The system should correctly calculate and save quiz results during normal operation.

NFR-04: Maintainability

The Python code should be organized clearly so that the application can be modified or extended easily.

NFR-05: Portability

The application should run on any system with Python 3 installed, without modification.

NFR-06: Data Persistence

Score history shall remain available after the application is closed and reopened.

9. Error Handling

The system shall handle common errors without crashing.

The system shall provide appropriate messages for:

- Missing or invalid question file
- Empty or missing score history file
- File read/write errors

After an error, the user should be able to return to the main menu and try again.

10. System Constraints

- The application shall be implemented in Python 3 using Tkinter.
- The user interface is a graphical desktop interface (GUI).
- Data shall be stored locally using JSON.
- No internet connection is required for normal operation.
- No external database is required.
- The system is intended for a small number of questions and score records.
- The project shall remain within the scope of a Python mini project.

11. Assumptions

- Python is installed on the development computer.
- The questions.json file is correctly formatted before the program runs.
- The user has permission to read and write files in the project directory.
- The user provides valid keyboard input eventually, after re-prompting.
- The storage files are accessible to the application.

12. Acceptance Criteria

The project shall be considered complete when:

- The application launches into a non-resizable GUI window.
- Questions load correctly from questions.json and display with radio buttons.
- The user cannot proceed to the next question without making a selection.
- The user can navigate back to earlier questions and retain previous choices.
- The score, percentage, and pass/fail indicator calculate accurately and display upon submission.
- Quiz attempts append properly to scores.json with timestamp, score, total, and percentage.
- The score history displays entries in reverse chronological order within a scrollable listbox.

13. Project Schedule and Milestones

The project is scheduled to be completed in a single working session (approximately 5 hours), broken down by task as shown in Figure 2. The schedule follows the SDLC sequence defined in this document: requirements review, design, implementation, testing, and reporting.

Task	9am	10am	11am	12pm	1pm
Load questions, set up project					
Build quiz loop (ask/answer)					
Score calculation & results display					
Score history save/load + testing					
Write report					

Figure 2: Development schedule

14. Requirement Summary

FR-01	Load questions	**FR-06**	View score history
FR-02	Run quiz	**FR-07**	Validate input
FR-03	Calculate score	**FR-08**	Navigation
FR-04	Display results	**NFR-01**	Usability
FR-05	Save score history	**NFR-02**	Performance
NFR-03	Reliability	**NFR-05**	Portability
NFR-04	Maintainability	**NFR-06**	Data persistence